

Problem Description

You are given an N-dimensional array A. You can execute an operation that removes the greatest and smallest elements from the array and replaces them with their difference. As a result, the array's size will drop by one after each operation. You are assigned Q jobs, each of which contains an integer K. After K operations, you must provide the total of all the items in the array for each task.

Constraints:

$$2 \leq N \leq 10^5$$

$$1 \leq Q \leq 10^5$$

$$0 \leq A[i] \leq 10^9$$

$$0 \leq K < N$$

Input:

First line contains two space-separated integers **N** and **Q**, denoting the number of elements in array and number of queries respectively.

Next line contains **N** space-separated integers denoting elements of the array.

Next **Q** lines contain a single integer **K**.

Output:

For each task, print answer in a new line.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5 2
3 2 1 5 4
1
2

EXPECTED OUTPUT

13
9

Test Case 2

INPUT (STDIN)

5 3
13 12 11 15 14
1
2
3

EXPECTED OUTPUT

43
35
15

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for(i=0;i<n;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

300

Test Case 3

NLOC

38

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 typedef struct {
6     long long val;
7     int id;
8 } Node;
9
10 typedef struct {
11     Node *data;
12     int size;
13     int is_min;
14 } Heap;
15
16 void push_heap(Heap *h, long long val, int id) {
17     int i = h->size++;
18     h->data[i].val = val;
19     h->data[i].id = id;
20     while (i > 0) {
21         int p = (i - 1) / 2;
22         int cond = h->is_min ? (h->data[i].val < h->data[p].val) : (h->data[i].val > h->data[p].val);
23         if (cond) {
24             Node tmp = h->data[i];
25             h->data[i] = h->data[p];
26             h->data[p] = tmp;
27             i = p;
28         } else {
29             break;
30         }
31     }
```

```

31 }
32}
33
34Node pop_heap(Heap *h) {
35     Node top = h->data[0];
36     h->data[0] = h->data[--h->size];
37     int i = 0;
38     while (2 * i + 1 < h->size) {
39         int left = 2 * i + 1;
40         int right = 2 * i + 2;
41         int best = left;
42         if (right < h->size) {
43             int cond = h->is_min ? (h->data[right].val < h->data[left].val) : (h->data[right].val > h->data[left].val);
44             if (cond) best = right;
45         }
46         int cond = h->is_min ? (h->data[best].val < h->data[i].val) : (h->data[best].val > h->data[i].val);
47         if (cond) {
48             Node tmp = h->data[i];
49             h->data[i] = h->data[best];
50             h->data[best] = tmp;
51             i = best;
52         } else {
53             break;
54         }
55     }
56     return top;

```

```

57}
58
59int main() {
60    int n, q;
61    if (scanf("%d %d", &n, &q) != 2) return 0;
62    long long *a = (long long*)malloc(n * sizeof(long long));
63    long long initial_sum = 0;
64    int i;
65    for(i=0;i<n;i++) {
66        if (scanf("%lld", &a[i]) != 1) a[i] = 0;
67        initial_sum += a[i];
68    }
69
70    int max_nodes = 2 * n + 5;
71    Heap min_h, max_h;
72    min_h.data = (Node*)malloc(max_nodes * sizeof(Node));
73    min_h.size = 0;
74    min_h.is_min = 1;
75
76    max_h.data = (Node*)malloc(max_nodes * sizeof(Node));
77    max_h.size = 0;
78    max_h.is_min = 0;
79
80    char *deleted = (char*)calloc(max_nodes, sizeof(char));
81

```

main | CH.SC.U4CSE25225 | Tree22.c

Ln 131, Col 1 Spaces: 4 UTF-8 CRLF C

```

82     for (i = 0; i < n; i++) {
83         push_heap(&min_h, a[i], i);
84         push_heap(&max_h, a[i], i);
85     }
86
87     long long *ans = (long long*)malloc(n * sizeof(long long));
88     ans[0] = initial_sum;
89
90     int next_id = n;
91     long long current_sum = initial_sum;
92
93     for (int step = 1; step < n; step++) {
94         while (min_h.size > 0 && deleted[min_h.data[0].id]) {
95             pop_heap(&min_h);
96         }
97         Node mn = pop_heap(&min_h);
98         deleted[mn.id] = 1;
99
100        while (max_h.size > 0 && deleted[max_h.data[0].id]) {
101            pop_heap(&max_h);
102        }
103        Node mx = pop_heap(&max_h);
104        deleted[mx.id] = 1;
105
106        long long diff = mx.val - mn.val;

```

```

106     long long diff = max_val - min_val;
107     current_sum -= 2 * mn.val;
108     ans[step] = current_sum;
109
110     int new_id = next_id++;
111     push_heap(&min_h, diff, new_id);
112     push_heap(&max_h, diff, new_id);
113 }
114
115 for (i = 0; i < q; i++) {
116     int k;
117     if (scanf("%d", &k) == 1) {
118         if (k >= 0 && k < n) {
119             printf("%lld\n", ans[k]);
120         }
121     }
122 }
123
124 free(a);
125 free(min_h.data);
126 free(max_h.data);
127 free(deleted);
128 free(ans);
129 return 0;
130 }
131

```


C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 2

3 2 1 5 4

1

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree22.exe

13

9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5 3

13 12 11 15 14

1

2

3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree22.exe

43

35

15

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>